



## **FACULTAD DE CIENCIAS DE LA VIDA Y TECNOLOGÍAS**

### **Integrantes:**

Litardo Hernandez Luis Antonio  
Steven Alexis Soledispa Espinaldez  
Ronnie Alexander Mera Cedeño

### **Asignatura:**

APLICACIONES WEB II

### **Curso:**

6to “A”

### **Periodo:**

2026-1

## Índice

|                                                                               |   |
|-------------------------------------------------------------------------------|---|
| 1. Tres problemas que enfrenté durante el desarrollo (Luis Litardo) .....     | 2 |
| 1.1 Conflictos al sincronizar el repositorio .....                            | 2 |
| 1.2 Escribir funciones en Go viniendo de otros lenguajes .....                | 2 |
| 1.3 Construir las rutas con Chi y entender cómo se forman las URLs .....      | 3 |
| 1.5 Reflexión sobre programar en Go .....                                     | 3 |
| 2. Tres problemas que enfrenté durante el desarrollo (Steven Soledispa) ..... | 4 |
| 2.1 Entender la diferencia entre PUT y PATCH .....                            | 4 |
| 2.2 Manejar 3 entidades relacionadas en un mismo módulo .....                 | 4 |
| 2.3 Organizar el código con subrouters de Chi .....                           | 4 |
| 2.4 Reflexión sobre programar en Go .....                                     | 4 |
| 3. Tres problemas que enfrenté durante el desarrollo (Ronnie Mera) .....      | 5 |
| 3.1 Validación de campos en los endpoints POST y PUT .....                    | 5 |
| 3.2 Organización de ramas y fusión al repositorio principal .....             | 5 |
| 3.3 Diseño de la relación entre las tres entidades del módulo .....           | 5 |
| 3.4 Reflexión sobre programar en Go .....                                     | 6 |

## **1. Tres problemas que enfrenté durante el desarrollo (Luis Litardo)**

### **1.1 Conflictos al sincronizar el repositorio**

Cuando intenté traer los cambios de mi compañero con `git pull`, me apareció un error que decía que no podía continuar porque tenía un archivo llamado `go.sum` de forma local que iba a ser sobrescrito. En ese momento no sabía qué hacer porque no entendía qué era ese archivo. Después de buscar un poco entendí que ese archivo lo genera Go automáticamente para guardar las versiones de las librerías del proyecto. Lo que hice fue eliminarlo y volver a hacer el pull, y funcionó. Fue algo simple pero me trabó un buen rato porque no lo había visto antes.

### **1.2 Escribir funciones en Go viniendo de otros lenguajes**

Lo que más me costó al escribir el código fue la forma en que Go declara las funciones. En otros lenguajes que he usado, una función se escribe de forma directa. Pero en Go, cuando una función pertenece a un struct, hay que agregar una parte extra antes del nombre que se llama receiver, por ejemplo `func (m *MemoriaSuscripciones) CrearServicio(...)`. Al principio no entendía para qué servía esa parte ni por qué a veces llevaba un asterisco y otras no. Estuve revisando el código de mi compañero varias veces hasta que le encontré la lógica y pude escribir mis propias funciones sin problema.

### 1.3 Construir las rutas con Chi y entender cómo se forman las URLs

Otro punto que me confundió fue cómo funcionan las rutas en Chi. En el archivo `main.go` se define un prefijo con `r.Mount`, y dentro del handler se definen las rutas más específicas. Lo que no tenía claro es que la URL final se forma combinando las dos partes. Por ejemplo, `/api/v1/suscripciones/servicios` no está escrita completa en ningún lado, sino que es la unión del prefijo del `main.go` con la ruta del handler. Cuando fui a probar en Postman no sabía qué URL poner porque pensaba que estaba definida en un solo lugar. Después de entender eso, todo fue más claro.

### 1.5 Reflexión sobre programar en Go

Antes de este proyecto ya había trabajado con varios lenguajes, así que programar en general no era algo nuevo para mí. Lo que sí fue nuevo fue adaptarme a la forma de pensar de Go. Es un lenguaje que no te deja ignorar los errores, siempre tienes que manejarlos con `if err != nil`, lo cual al inicio se siente como que escribes mucho código para cosas pequeñas, pero después entiendes que eso hace que el programa sea más seguro y predecible.

También me gustó cómo Go organiza los proyectos. Tener carpetas separadas para los modelos, los handlers y el storage hace que sea fácil saber dónde está cada cosa. No es como otros lenguajes donde puedes poner todo junto en un solo archivo.

En resumen, Go me pareció un lenguaje diferente pero interesante. Cuesta un poco al principio acostumbrarse a su forma de escribir el código, pero una vez que le agarras el ritmo, se trabaja bien.

## **2. Tres problemas que enfrenté durante el desarrollo (Steven Soledispa)**

### **2.1 Entender la diferencia entre PUT y PATCH**

Al principio no tenía claro por qué necesitaba dos formas de actualizar una orden. Que PUT reemplaza todo el recurso (necesita todos los campos) mientras que PATCH solo modifica algunos campos específicos (estado y diagnóstico), y que el storage necesitaba una función distinta para cada caso.

### **2.2 Manejar 3 entidades relacionadas en un mismo módulo**

Mi módulo no era solo una entidad (órdenes correctivas) sino tres relacionadas (órdenes, procesos, evidencias) según lo que realice en el diagrama ER del Hito 1. Esto significó triplicar la lógica de storage y handlers manteniendo el mismo patrón para cada una, sin mezclar los datos ni los contadores de ID.

### **2.3 Organizar el código con subrouters de Chi**

Inicialmente se registraba las rutas directamente en main.go, pero la guía pedía usar subrouters con r Mount. Debido a esto se tuvo que reestructurar el código para que CorrectivosRouter devolviera su propio router y main.go solo lo montara, entendiendo cómo Chi anida rutas (/api/v1/correctivos + /ordenes = /api/v1/correctivos/ordenes).

### **2.4 Reflexión sobre programar en Go**

Programar este módulo en Go fue una experiencia distinta a lo que había hecho antes. Al principio me costó acostumbrarme a lo estricto que es el lenguaje: si declaras una variable y no la usas, no compila. Si te falta manejar un error, Go no te deja pasarlo por alto tan fácil como en otros lenguajes. Eso al principio me pareció tedioso, pero con el tiempo entendí que es justamente lo que hace que el código sea más confiable, porque te obliga a pensar en los casos de error desde el principio y no después.

También entendí mejor cómo funciona la concurrencia con el sync.Mutex. No había trabajado antes con la idea de que dos peticiones puedan llegar al mismo tiempo y modificar los mismos datos, pero al usar Lock() y Unlock() en cada función del storage, entendí por qué es importante proteger los datos compartidos.

Por último, trabajar con structs y JSON tags me hizo ver cómo Go conecta directamente el modelo de datos con lo que se envía y recibe por la API, sin pasos intermedios complicados. En general, Go me pareció un lenguaje que exige más disciplina al escribir, pero que a cambio da más seguridad de que el código va a funcionar como esperas.

### **3. Tres problemas que enfrenté durante el desarrollo ( Ronnie Mera)**

#### **3.1 Validación de campos en los endpoints POST y PUT**

Uno de los retos al implementar los endpoints de creación y actualización fue decidir qué campos debían ser obligatorios para cada una de las tres entidades (MantenimientoPreventivo, TareaPreventiva e InsumoPreventivo), y qué debía ocurrir si el cliente enviaba un JSON incompleto, con valores vacíos o con tipos de datos inválidos. Sin estas validaciones, la API aceptaría registros incoherentes (por ejemplo, una tarea sin descripción o un insumo con costo negativo). Solución: en cada handler de creación y actualización se agregó una validación previa al guardado: se revisa que los campos de texto obligatorios (como Equipo, Descripcion o Nombre) no estén vacíos usando strings.TrimSpace, y que los valores numéricos relevantes (como Duracion o Costo) sean mayores a cero. Si alguna validación falla, se responde con código 400 (Bad Request) y un mensaje descriptivo del error; si el JSON recibido no se puede decodificar, también se responde 400. Esto permitió entender la importancia de validar los datos en el servidor antes de almacenarlos, sin depender únicamente de lo que el cliente envíe.

#### **3.2 Organización de ramas y fusión al repositorio principal**

Al trabajar en equipo sobre un mismo repositorio, surgió la duda de cómo aislar el trabajo individual sin afectar el código de los demás integrantes. Existía incertidumbre sobre si al crear una rama propia se perderían los archivos ya existentes de los compañeros. Solución: se creó la rama “feature/modulo-preventivo” a partir de la rama master actualizada, lo que permitió conservar todo el código existente (módulos correctivo y de suscripciones) y agregar únicamente los archivos nuevos del módulo preventivo. Una vez probado y funcionando, se subió la rama con “git push” y se integró al master mediante un Pull Request en GitHub. Esto permitió comprender el flujo de trabajo colaborativo con control de versiones: cada integrante desarrolla en su propia rama y luego se fusiona al código principal.

#### **3.3 Diseño de la relación entre las tres entidades del módulo**

El módulo preventivo requería definir tres entidades relacionadas entre sí (MantenimientoPreventivo, TareaPreventiva e InsumoPreventivo), y era necesario que la lógica tuviera sentido dentro del negocio: una orden de mantenimiento puede tener varias tareas y consumir varios insumos. Solución: se definió que cada TareaPreventiva e InsumoPreventivo almacena el campo MantenimientoPreventivoID, que actúa como

llave de relación hacia el mantenimiento al que pertenecen. Esto refleja un flujo real: primero se crea el mantenimiento programado para un equipo, luego se registran las tareas a realizar (con su duración estimada) y finalmente los insumos utilizados (con su costo). Cada entidad se implementó con su propio CRUD completo siguiendo el mismo patrón usado en los módulos correctivo y de suscripciones del proyecto.

### **3.4 Reflexión sobre programar en Go**

Trabajar con Go para este módulo permitió comprender de forma práctica varios conceptos del lenguaje. La sintaxis explícita de Go, especialmente el manejo de errores mediante el patrón “valor, error” y el patrón “comma-ok” (valor, bool) usado en las funciones de búsqueda del storage, obliga a pensar en todos los casos posibles (encontrado / no encontrado, válido / inválido) antes de continuar con la lógica. El uso de sync.Mutex en la capa de almacenamiento en memoria ayudó a entender la importancia de proteger datos compartidos frente a accesos concurrentes, algo fundamental en un servidor HTTP que puede recibir múltiples peticiones al mismo tiempo. Asimismo, la organización del proyecto en paquetes (models, storage, handlers) mostró cómo Go favorece una separación clara de responsabilidades, lo cual facilita que varios integrantes trabajen en módulos distintos sin generar conflictos graves en el código. En conjunto, este proyecto permitió no solo practicar la creación de una API REST con Go y el framework chi, sino también reforzar habilidades de trabajo colaborativo con Git y GitHub, resolución de conflictos de sincronización y documentación técnica del trabajo realizado.